

72.07 - Protocolos de Comunicación

Proyecto Especial



Integrantes:

- Asin, Gerónimo - 65486
- Garbarz, Andrés Matías - 65488
- Lee, Sebastián Nicolás - 64675
- Lifischtz, Santiago - 65782

Grupo: 16

Fecha de entrega: 14/07/2026

Índice

Índice.....	1
1. Abstracto.....	3
2. Protocolos y Aplicaciones.....	3
2.1 Servidor Proxy SOCKSv5.....	3
2.2 Protocolo de Monitoreo - ChungusMonitor.....	5
2.2.1 Transporte y diseño general.....	5
2.2.2 Framing.....	6
2.2.3 Establecimiento de la sesión.....	6
2.2.4 Autenticación.....	6
2.2.5 Comandos.....	8
Los errores utilizan mensajes específicos:.....	9
2.2.6 Parámetros configurables.....	9
2.2.7 Pipelining.....	10
2.2.8 Cierre de la conexión.....	10
2.2.9 Límites propuestos.....	10
2.3 Aplicación Cliente.....	11
3. Proceso de Desarrollo.....	12
3.1 Echo Server Bloqueante.....	13
3.2 Echo Server No Bloqueante.....	13
3.3 Implementación de SOCKSv5.....	13
3.4 Desarrollo del Protocolo de Administración y Aplicación Cliente.....	14
4. Problemas Encontrados.....	14
5. Limitaciones.....	16
6. Posibles Extensiones.....	16
7. Conclusiones.....	18
8. Guía de Instalación.....	19
8.1 Requisitos del Entorno.....	19
8.2 Obtención del Código y Compilación.....	19
8.3 Entorno Docker.....	20
8.4 Artefactos Generados.....	20
9. Guía de Configuración.....	20
9.1 Argumentos del Servidor.....	20
9.2 Argumentos del Cliente de Monitoreo.....	21

9.3 Parámetros en Tiempo de Ejecución.....	21
9.4 Usuario Inicial y Roles.....	22
10. Ejemplos de Prueba.....	22
10.1 Pruebas Automatizadas.....	22
10.2 Pruebas Funcionales del Proxy SOCKS.....	23
Destino por FQDN (resolución DNS en el servidor).....	23
Destino por FQDN resuelto por curl (el proxy recibe la IP).....	23
Credenciales inválidas: curl reporta el rechazo de autenticación.....	23
Destino inexistente: el proxy responde host inalcanzable (REP 0x04).....	23
Puerto cerrado: el proxy responde conexión rechazada (REP 0x05).....	24
10.3 Pruebas de Estrés y Throughput.....	24
11. Ejemplos de Configuración y Monitoreo.....	26
Gestion de usuarios.....	26
Registro de Accesos y Sesiones Activas.....	27
Cambios de Configuración en Tiempo de Ejecución.....	28
12. Documento de Diseño del Proyecto.....	29
14. Repositorio.....	31
15. Bibliografía.....	32

1. Abstracto

El presente documento detalla el proceso de desarrollo de un servidor proxy que implementa el protocolo SOCKSv5, capaz de atender a todos sus clientes desde un único hilo mediante E/S no bloqueante multiplexada, dentro del marco de la materia Protocolos de Comunicación en el Instituto Tecnológico de Buenos Aires. Además, se implementó un protocolo de administración custom (ChungusMonitor) que permite monitorear y configurar el proxy en tiempo de ejecución.

2. Protocolos y Aplicaciones

El sistema consta de un único proceso que ofrece dos servicios TCP:

Servicio	Puerto Default	Rol
SOCKSv5	1080	Proxy de clientes
ChungusMonitor	8080	Configuraciones y monitoreo

Ambos conviven en el mismo selector de I/O y comparten un struct de estado (**monitor_store**), de modo que lo expuesto por el puerto de monitoreo refleja el estado real del proxy.

2.1 Servidor Proxy SOCKSv5

El servidor al inicializarse, crea un socket pasivo IPv6 dual-stack, capaz de aceptar tanto conexiones IPv4 como IPv6 sobre un mismo descriptor. Para ello se desactiva la opción IPV6_V6ONLY y se habilita SO_REUSEADDR, tras lo cual el socket se registra en el selector con interés de lectura para comenzar a aceptar nuevas conexiones. Cada cliente aceptado se representa mediante una sesión que contiene dos buffers cruzados de 4096 bytes *c2o* (cliente hacia origen) y *o2c* (origen hacia cliente) junto con una máquina de estados que modela las distintas etapas del protocolo.

El valor default de 4096 bytes fue elegido por ser también el default de nginx (4k u 8k, dependiendo de la plataforma). De todos modos, el tamaño del buffer es configurable en runtime. Cambiar este valor mientras hay sesiones activas no cambia el tamaño de sus buffers, sino que solo las nuevas conexiones utilizan el nuevo tamaño. Esta decisión es solo una simplificación para evitar todo tipo de conflicto que aparece al achicar un buffer que está en uso.

La primera etapa corresponde a la negociación del método de autenticación. Un parser consume el mensaje inicial del cliente byte a byte y verifica los métodos ofrecidos. Si entre ellos se encuentra el método usuario/contraseña (**0x02**), el servidor responde con los bytes **[05][02]** y continúa hacia la fase de autenticación. Si el cliente no ofrece (**0x02**) pero sí el método sin autenticación (**0x00**), el servidor acepta y procede a registrar al usuario como anónimo (username: Anon). En caso contrario, responde **[05][FF]** y cierra la conexión una vez enviada la respuesta. GSSAPI no se implementa, como especifica la consigna.

La autenticación se implementa conforme al RFC 1929. En esta etapa, un segundo parser interpreta la subnegociación de usuario y contraseña y valida las credenciales contra la tabla de usuarios del **monitor_store**, utilizada también por el protocolo de monitoreo. Cuando las credenciales son válidas, el servidor responde **[01][00]**; si son inválidas o el mensaje recibido está mal formado, responde **[01][01]** y finaliza la conexión.

Superada la autenticación, el servidor procesa el pedido de conexión (**CONNECT**) mediante un tercer parser encargado de interpretar el request definido en el RFC 1928. Se soportan los tres tipos de dirección especificados por el protocolo (IPv4, IPv6 y FQDN), aunque únicamente se acepta el comando **CONNECT**, rechazándose los comandos **BIND** y **UDP ASSOCIATE**.

Cuando el destino se especifica mediante un nombre de dominio (FQDN), la resolución DNS se realiza mediante la llamada bloqueante **getaddrinfo(3)** ejecutada en un hilo dedicado, creado exclusivamente para esa operación y descartado una vez finalizada. Cuando el hilo termina, guarda el resultado en una lista y escribe un `uint64_t` en un *eventfd* para despertar al selector. En la siguiente vuelta del event loop, el selector revisa la lista de trabajos terminados y llama al callback `handle_block` del socket correspondiente, así la máquina de estados sigue en el hilo principal. Esto reemplaza la implementación vieja con señales (**SIGUSR1**), que era problemática por la reentrancia y porque interrumpía `epoll_wait`.

La conexión con el servidor de origen se establece mediante un **connect(2)** no bloqueante. El descriptor correspondiente se registra con interés de escritura y es el kernel quien notifica posteriormente si la conexión se completó o falló, información que se obtiene mediante **getsockopt(SO_ERROR)**. Si la resolución DNS produjo múltiples direcciones y la conexión falla, ya sea de manera inmediata o asincrónica, el servidor reintenta automáticamente utilizando la siguiente entrada de la lista de `addrinfo`; por ejemplo, si una dirección IPv6 no resulta accesible, se intenta posteriormente con una dirección IPv4.

Una vez determinado el resultado del intento de conexión, el proxy responde al cliente utilizando los códigos de estado definidos por el protocolo. Se devuelve **0x00** cuando la conexión se establece correctamente, **0x01** ante fallos generales del servidor, como mensajes mal formados o falta de recursos, **0x04** cuando la resolución DNS no logra alcanzar el host solicitado y **0x05**

cuando todas las direcciones obtenidas durante la resolución fallan al intentar establecer la conexión.

Además, antes de iniciar la conexión con el destino, el servidor consulta la lista de denegaciones (ACL) del `monitor_store`. Si el nombre de host o la dirección IP resuelta figura en la lista, el proxy responde con el código SOCKS 0x02 (connection not allowed by ruleset) y finaliza la

sesión sin intentar el connect. El chequeo se realiza tanto para destinos FQDN (con la lista de hosts) como para destinos IP literales o resueltos por DNS (con la lista de direcciones).

Luego de enviar esta respuesta, y siempre que la conexión haya sido exitosa, el servidor entra en la etapa de relay, donde realiza la copia bidireccional de datos entre cliente y origen. Los bytes leídos desde el cliente se almacenan en el buffer `c2o` y posteriormente se envían al servidor de origen, mientras que los datos recibidos desde el origen siguen el camino inverso a través del buffer `o2c`. Los intereses de lectura y escritura de ambos sockets se recalculan dinámicamente según exista espacio disponible o datos pendientes en cada buffer, implementando de manera natural el control de flujo. En consecuencia, si el servidor de origen deja de consumir datos, el proxy deja temporalmente de leer información proveniente del cliente.

El servidor también implementa un timeout de inactividad configurable (parámetro `timeout`, por defecto deshabilitado). Cada sesión registra el timestamp de su última actividad (`last_activity`). Periódicamente, en cada vuelta del event loop, una función de expiración recorre una lista doblemente enlazada ordenada por antigüedad y cierra las sesiones que superen el límite configurado. Si la sesión aún no entró en relay (por ejemplo, quedó idle durante el handshake o la conexión al origen), el proxy responde al cliente SOCKS con el código 0x06 (TTL expired) del RFC y cierra tras vaciar el buffer de salida. Si ya estaba en relay, se cierra la sesión sin enviar ese reply, porque la respuesta CONNECT ya fue entregada y el canal está en transferencia de datos; el cierre queda registrado en el access log como `TTL_EXPIRED`.

El servidor ofrece una blacklist configurable, tanto para nombres de dominio como para direcciones IP. Internamente, son dos listas que bloquean el acceso a ciertos destinos, una con nombres, cuya única restricción es que no pueden ser una IP válida, y otra que exclusivamente tiene direcciones IP, que pueden ser IPv4 o IPv6. Al intentar iniciar una conexión, si el FQDN pedido está dentro de la primera lista, la conexión se rechaza de inmediato. Si no, resuelve con DNS y luego verifica si la IP destino está en la segunda lista, también rechazando la conexión, al igual que si se intentara conectar a una IP directamente.

Durante todo el ciclo de vida de la sesión, cada transición de estado se notifica al **monitor_store**, incluyendo la aceptación de la conexión, la autenticación del usuario, el establecimiento de la conexión con el destino, la cantidad de bytes transferidos y el cierre o fallo de la sesión. Esta

información alimenta las métricas del sistema, la lista de conexiones activas y el registro de accesos expuesto por el protocolo de monitoreo.

2.2 Protocolo de Monitoreo - ChungusMonitor

Esta subsección especifica el protocolo diseñado para el monitoreo y la configuración del servidor. La descripción es agnóstica al lenguaje de implementación y contiene los detalles necesarios para implementar un cliente o servidor compatible.

2.2.1 Transporte y diseño general

ChungusMonitor es un protocolo **de texto**, orientado a conexión, que corre sobre **TCP**. El servidor escucha en un puerto dedicado (8080 por defecto) distinto al del servicio SOCKS. Se eligió un formato de texto línea a línea — en la línea de protocolos como POP3 o el protocolo de Redis — por su simplicidad de implementación y depuración: cualquier problema puede diagnosticarse observando el tráfico directamente, y no se requiere serialización binaria ni consideraciones de endianness.

2.2.2 Framing

Cada mensaje, ya sea un comando o respuesta, es una línea de texto terminada en `\n`. El servidor también acepta el terminador `\r\n`, descartando el `\r`. La longitud máxima de una línea de comando es de **512 bytes**. Si un cliente la excede, el servidor responde: **-ERR line too long** y descarta lo acumulado. Los argumentos de un comando se separan con espacios simples, permitiendo hasta 8 tokens por comando.

2.2.3 Establecimiento de la sesión

Al aceptar una conexión, el servidor envía inmediatamente el saludo:

```
+OK ChungusMonitor v1.0
```

```
.
```

Toda respuesta del servidor comienza con **+OK** (éxito) o **-ERR** (error) seguido de texto descriptivo. Un comando puede producir múltiples líneas de respuesta (por ejemplo **CONNECTIONS** emite una línea **+OK** por sesión activa).

También, todas las respuestas, finalizan con un línea que contiene únicamente un punto (`.`), actuando como delimitador para el fin de la respuesta. Esto permite al cliente distinguir el final de una respuesta multilínea como el **CONNECTIONS**.

2.2.4 Autenticación

La sesión atraviesa una máquina de estados de dos fases:

En el estado inicial solo se acepta el comando **AUTH**. Cualquier otro comando produce el error: **-ERR not authenticated**.

La sentencia: **AUTH *usuario contraseña*** valida las credenciales contra la tabla de usuarios compartida con **SOCKS** y exige que el usuario tenga **rol administrador**.

Las respuestas posibles son:

- **+OK** (éxito)
- **-ERR invalid credentials** (credenciales incorrectas)
- **-ERR not admin** (credenciales válidas pero sin rol de administrador).

Una vez autenticado, un **AUTH** repetido produce **-ERR already authenticated**.

La decisión de autenticar dentro del propio protocolo (en lugar de, por ejemplo, restringir por dirección IP) reutiliza la tabla de usuarios existente y permite distinguir roles: los usuarios comunes pueden usar el proxy SOCKS pero no administrar el servidor.

2.2.5 Comandos

Comando	Sintaxis esperada	Descripción
AUTH	AUTH usuario contraseña	Autentica la sesión como administrador. Obligatorio antes de cualquier otro comando.
STATS	STATS	Devuelve las métricas globales.
CONNECTIONS	CONNECTIONS	Lista las sesiones SOCKS activas.
USERS	USERS	Lista todos los usuarios registrados en el servidor, indicando su rol (admin o user).
CONFIG	CONFIG parámetro valor	Modifica un parámetro de configuración en tiempo de ejecución.
	CONFIG parámetro	Lee el valor actual del parámetro. Responde +OK <valor>.
ACCESS_LOG	ACCESS_LOG [usuario]	Devuelve el registro de accesos, del más reciente al más antiguo, opcionalmente filtrado por usuario.
ADD_USER	ADD_USER usuario contraseña [admin]	Crea un usuario; el sufijo admin le otorga rol administrador.
DEL_USER	DEL_USER usuario	Elimina un usuario. Se rechaza si es el último administrador.
SET_PASSWORD	SET_PASSWORD usuario contraseña	Cambia la contraseña de un usuario existente.
DENY_HOST	DENY_HOST hostname	Deniega el acceso del proxy a un nombre de host (FQDN).

Comando	Sintaxis esperada	Descripción
DENY_IP	DENY_IP dirección	Deniega el acceso del proxy a una dirección IPv4 o IPv6.
UNDENY	UNDENY hostname-o-dirección	Elimina una regla de denegación existente.
DENY_LIST	DENY_LIST [host ip]	Lista las reglas de denegación activas. Opcionalmente filtrado por tipo.
QUIT	QUIT	Cierra la sesión.

Los errores utilizan mensajes específicos:

- -ERR syntax error
- -ERR unknown command
- -ERR unknown param
- -ERR invalid value
- -ERR user exists
- -ERR user not found
- -ERR user table full
- -ERR cannot delete last admin
- -ERR not authenticated
- -ERR already authenticated
- -ERR line too long
- -ERR host already denied
- -ERR address already denied
- -ERR rule not present
- -ERR failed to add rule

2.2.6 Parámetros configurables

El comando CONFIG acepta los siguientes parámetros, con validación de rango en el servidor:

Parámetro	Efecto	Rango válido
timeout	Segundos de inactividad tras los cuales se cierra una sesión SOCKS (0 = deshabilitado)	0 – 86400
max_connections	Tope de clientes SOCKS simultáneos	1 – 16384
io_buffer_size	Tamaño de buffer de E/S para sesiones nuevas	1024 – 65536

2.2.7 Pipelining

El servidor soporta *pipelining*: un cliente puede enviar varios comandos en un mismo segmento TCP (o incluso un comando fragmentado en varios segmentos) y el comportamiento es idéntico al de recibirlos de a uno. Esto se logra acumulando los bytes recibidos en un buffer y alimentando el intérprete de líneas byte a byte, sin asumir jamás que una lectura del socket corresponde a un mensaje completo.

2.2.8 Cierre de la conexión

Existen dos formas de terminar una sesión:

1. **QUIT**: El servidor encola +OK y marca la sesión para cierre; el descriptor se cierra recién cuando el buffer de salida se vació, garantizando que el cliente reciba todas las respuestas pendientes.
2. **Half-close**: El cliente puede cerrar su lado de escritura después de enviar sus comandos. Al detectar el fin de archivo, el servidor procesa todos los comandos pendientes en el buffer, incluida una eventual última línea sin \n terminador, envía las respuestas y recién entonces cierra la conexión.

2.2.9 Límites propuestos

Límite	Valor
Longitud máxima de línea	512 bytes
Longitud máxima de nombre de usuario	64 caracteres

Límite	Valor
Longitud máxima de contraseña	255 caracteres
Cantidad máxima de usuarios	256
Capacidad del registro de accesos	4096 entradas (buffer circular: las más antiguas se sobrescriben)
Longitud máxima de host de destino registrado	255 caracteres

2.3 Aplicación Cliente

Para facilitar la administración del servidor se desarrolló bin/client, una aplicación con interfaz de terminal (TUI) que implementa una navegación interactiva por menús sobre el protocolo ChungusMonitor. Utiliza códigos de escape ANSI para el renderizado de menús y el modo raw del terminal (tcsetattr con ICANON y ECHO desactivados) para capturar las teclas de dirección (flechas arriba y abajo) y Enter sin necesidad de *ncurses* ni libs externas.

La invocación general es:

```
./bin/client [-p <puerto>]
```

Al conectarse, la aplicación presenta un formulario de inicio de sesión donde el operador ingresa su nombre de usuario y contraseña (la contraseña se ingresa con ECHO desactivado). Si la autenticación falla, el formulario se reinicia; esto ocurre en loop hasta que sea exitosa o presione q/Esc, que la conexión se cierra.

Una vez autenticado, se despliega un menú principal con las siguientes opciones navegables con las teclas flecha arriba y abajo:

Menu	Descripción
Stats	Métricas del proxy (bytes transferidos, sesiones)
Active connections	Lista de sesiones SOCKS activas
Registered users	Lista de usuarios registrados

Menu	Descripción
Configuration	Ajuste de parámetros runtime
Add user	Creación de un nuevo usuario
Delete user	Eliminación de un usuario existente
Change password	Cambio de contraseña de un usuario
Access log	Historial de accesos
Deny host	Agrega un hostname a la lista de hosts denegados
Deny IP	Agrega una dirección IP a la lista de IPs denegadas
Undeny	Elimina una regla de denegación existente
Deny list	Muestra las reglas de denegación activas
Quit	Cierre de sesión y salida

Cada opción del menú ejecuta el comando correspondiente del protocolo ChungusMonitor (Sección 2.2) y muestra el resultado formateado en pantalla. La opción Configuration abre un sub-menú donde se selecciona el parámetro a modificar (timeout, max_connections, io_buffer_size) y se ingresa el nuevo valor numéricamente.

3. Proceso de Desarrollo

El proyecto se abordó de manera incremental: cada etapa produjo un servidor funcional y verificable antes de pasar a la siguiente. Esta estrategia, recomendada por la cátedra, evitó el

riesgo de implementar primero una versión bloqueante del proxy completo y tener que cambiarla luego al modelo no bloqueante — una migración notoriamente costosa. Los módulos provistos por la cátedra (selector, buffer, máquina de estados y parser) se integraron desde el comienzo y estructuraron todo el diseño posterior.

3.1 Echo Server Bloqueante

La primera etapa consistió en un servidor de eco elemental: un socket pasivo TCP que acepta una conexión, lee datos y los devuelve. Su propósito fue afianzar el manejo de la API de sockets de Berkeley (socket, bind, listen, accept, read, write) y de condiciones como el fin de archivo (read == 0). Esta versión, deliberadamente bloqueante, sirvió como punto de partida conceptual y no forma parte del producto final.

3.2 Echo Server No Bloqueante

El paso decisivo fue reescribir el eco sobre el selector de la cátedra: un único hilo que multiplexa múltiples descriptores y despacha manejadores de lectura y escritura cuando el kernel indica que una operación no bloqueará. Cada conexión pasó a tener su propio buffer, y los intereses de cada socket (lectura si hay espacio en el buffer, escritura si hay datos pendientes) se recalculan tras cada evento. Esta versión, conservada en `src/server/echo/echo.c` con sus pruebas automatizadas, estableció el patrón estructural — socket pasivo registrado en el selector, sesión con buffer propio, manejadores por descriptor — que se replicó luego tanto en el proxy SOCKS como en el servidor de monitoreo. El selector utiliza `epoll(7)` como mecanismo de multiplexación subyacente, y la notificación de tareas bloqueantes (como la resolución DNS) se implementa con `eventfd(2)`, evitando los problemas de seguridad asociados al uso de señales POSIX para este fin.

3.3 Implementación de SOCKSv5

Sobre esa base se construyó el proxy en fases sucesivas. Primero se incorporó la conexión saliente no bloqueante hacia un origen y el camino de escritura con buffers por socket, manejando escrituras parciales. Luego se implementaron los tres parsers del protocolo — negociación de métodos, autenticación RFC 1929 y request CONNECT — como módulos independientes sin entrada/salida, cada uno con sus pruebas unitarias, lo que permitió validarlos alimentándolos byte a byte sin necesidad de sockets. La máquina de estados de la cátedra integró estas piezas: cada estado del protocolo tiene sus manejadores y las transiciones se derivan del resultado de los parsers. Finalmente se agregaron la resolución DNS en un hilo auxiliar con notificación al selector, el failover entre direcciones y los códigos de error específicos. La validación de extremo a extremo se realizó con:

```
curl --socks5 admin:admin@localhost:1080 http://example.com
```

3.4 Desarrollo del Protocolo de Administración y Aplicación Cliente

Con el proxy operativo se diseñó el protocolo ChungusMonitor (sección 2.2) y se implementó en dos capas deliberadamente separadas: **monitor_commands.c** contiene el intérprete de líneas y los manejadores de comandos **sin ningún socket** — recibe bytes y escribe respuestas en un buffer —, mientras que **monitor.c** aporta la capa de red (socket pasivo, sesiones, manejadores del selector). Esta separación permitió probar toda la lógica del protocolo de forma unitaria, incluyendo pipelining y half-close, sin depender de la red. El estado compartido se centralizó en el módulo **store** (**monitor_store**): la autenticación SOCKS y los comandos de administración operan sobre la misma tabla de usuarios, y el proxy reporta allí métricas, sesiones y accesos. Por último se desarrolló la aplicación cliente (sección 2.3) y se completó el apagado ordenado del servidor ante SIGINT y SIGTERM.

Posteriormente, la aplicación cliente se refactorizó de un REPL de línea de comandos a una interfaz de terminal interactiva (TUI), organizada en tres módulos: **protocol.c** (comunicación con el servidor), **ui.c** (manejo del terminal, menús e ingreso de datos) y **main.c** (pantallas de cada función y bucle principal). El servidor de monitoreo eliminó el comando **HELP**, ya que la TUI provee su propia guía de navegación.

4. Problemas Encontrados

Durante el desarrollo surgió la dificultad de mapear fallos reales de red a los códigos de respuesta (REP) definidos por el RFC 1928. El estándar enumera ocho valores distintos —desde éxito (**0x00**) hasta tipos de dirección no soportados (**0x08**)—, pero en la práctica de un proxy que solo admite **CONNECT** sobre TCP la mayoría son poco frecuentes o directamente inaplicables. El código **0x06** (TTL expired), por ejemplo, está pensado en el RFC para el reenvío de datagramas en **UDP ASSOCIATE**; en esta implementación se reutiliza para el timeout de inactividad configurable cuando la sesión aún no entró en relay. Ante esta disparidad entre el catálogo del estándar y el tráfico real, se priorizaron los casos verificables con herramientas estándar: **0x04** cuando falla la resolución DNS, **0x05** cuando ninguna dirección devuelta acepta la conexión, y **0x01** para los demás fallos (mensajes mal formados, falta de recursos). Los comandos **BIND/UDP ASSOCIATE** y los **ATYP** no soportados se responden con **0x07** y **0x08** respectivamente, conforme al RFC.

Otro punto de confusión fue la distinción entre el parámetro **timeout** configurable en ChungusMonitor y el **TTL** de registros DNS. El comando **CONFIG timeout** mide inactividad de una sesión SOCKS ya establecida (por defecto deshabilitado con valor 0) y se evalúa

comparando la última actividad con la hora actual; no influye en la resolución de nombres ni reimplementa la caducidad de entradas en caché. Dado que cada FQDN se resuelve con **getaddrinfo** en el momento del pedido —sin caché intermedia—, el TTL de los registros lo gestiona únicamente el resolvidor del sistema operativo y no constituyó un problema de implementación en sí.

Sobre la transición de señales a `eventfd`, la implementación original usaba `SIGUSR1` para que el hilo de consulta DNS notificara al selector. Esto requería `pthread_kill` con `sigaction` y manejos de `EINTR` en `epoll_wait`, con riesgos de que entre la señal en cualquier momento. Se reemplazó por un `eventfd` registrado en `epoll`, entonces ahora el hilo DNS escribe un valor, `epoll_wait` retorna normalmente y el selector procesa la cola de trabajos (distinguiendo los FDs de evento (DNS) y los de I/O). Esto eliminó todo uso de señales entre hilos, por lo que `sigaction` solo queda para `SIGINT/SIGTERM`. Cabe destacar que el uso de `epoll` (reemplazando el `select`) es clave para la escalabilidad con cientos de conexiones. Con `select` hay que iterar sobre todos los descriptores registrados en cada llamada, mientras que `epoll_wait` solo entrega los que están listos, evitando caídas de performance a medida que crece la concurrencia.

Otro problema encontrado durante las pruebas fue la diferencia entre los flags `--socks5` y `--socks5-hostname` de `curl`. Con `--socks5`, `curl` resuelve el nombre de host localmente y envía al proxy la dirección IP ya resuelta, por lo que el servidor recibe un ATYP IP en lugar de un ATYP FQDN y el chequeo se realiza contra la lista de IPs en lugar de la lista de hosts. Para que el proxy resuelva el nombre y aplique el chequeo de hosts es necesario usar `--socks5-hostname`, que envía ATYP FQDN. Ambos modos son correctos según el protocolo SOCKS —la diferencia es propia de `curl`— pero durante el desarrollo costó identificar por qué el bloqueo no funcionaba correctamente, pero era porque algunos estábamos testeando con `--socks5`.

También relacionado a lo último, la lista de hosts denegados utiliza comparación exacta de cadenas (`strcmp`). Denegar `google.com` no bloquea `www.google.com` ni `mail.google.com`, ya que son strings distintos. Hace lo que especificamos, pero para el administrador resulta incómodo tener que agregar cada subdominio por separado.

Por último, la interacción entre E/S no bloqueante, **`connect()`** asíncronico y failover entre direcciones exigió un cuidado extra al propagar errores: un fallo inmediato en una entrada de **`addrinfo`** no es equivalente al rechazo reportado por **`getsockopt(SO_ERROR)`** tras un evento de escritura en el socket de origen, y ambos debían converger en el mismo mecanismo de reintento sin bloquear el hilo del selector. La resolución DNS, al ser la única operación bloqueante permitida, reforzó la separación entre lógica de protocolo (parsers sin E/S) y manejo de red: los parsers se probaron alimentándolos byte a byte, incluido pipelining de greeting, autenticación y request en un solo segmento, antes de integrar la máquina de estados completa.

5. Limitaciones

La primera limitación del proyecto concierne a la cantidad máxima de conexiones SOCKS simultáneas. Ese tope lo fija el parámetro **max_connections** del **monitor_store** (1024 por defecto), comprobado al aceptar cada cliente: si el cupo está lleno, la conexión entrante se cierra sin registrarla en el selector. En fase de relay, cada sesión activa utiliza dos descriptors registrados en el multiplexor (socket del cliente y socket hacia el origen), además de los sockets pasivos de SOCKS y monitoreo y de las conexiones de administración. El selector se crea con **selector_new(1024)** como pre-dimensionado inicial de su tabla interna y del buffer de eventos de **epoll**, pero puede crecer dinámicamente al registrar descriptors de número más alto; no constituye, por sí solo, el límite principal de concurrencia. Complementariamente, el arreglo **sessions[]** —donde se rastrean las sesiones para métricas, el comando **CONNECTIONS** y el registro de accesos— arranca dimensionado al **max_connections** inicial y, si ese parámetro se eleva mediante **CONFIG**, se redimensiona hasta el nuevo valor (sin superar 16384). No se permite bajar **max_connections** por debajo de las sesiones SOCKS ya abiertas.

Otra limitación importante es que no se implementó el soporte para los comandos **BIND** ni **UDP ASSOCIATE** del protocolo SOCKSv5. Solo se admite **CONNECT**. Se decidió dejar fuera del alcance la gestión de sockets UDP asociados, el reenvío de datagramas sin conexión previa y las asociaciones de escucha exigidas por **BIND**, por la complejidad que agregan y porque no son requisitos centrales de la consigna.

Por último, la tabla de usuarios (hasta 256 entradas), las métricas y el registro de accesos (buffer circular de 4096 entradas) residen exclusivamente en memoria: un reinicio del servidor los pierde. Los usuarios iniciales deben volver a indicarse con **-u** y **-a** al arrancar; no existe un usuario **admin/admin** hardcodeado. La consigna admite métricas volátiles, pero la falta de persistencia de usuarios, ACL, configuración y log constituyen limitaciones operativas.

6. Posibles Extensiones

En el plano funcional del proxy SOCKS quedan pendientes los comandos **UDP ASSOCIATE** y **BIND** del RFC 1928, que exigirían extender la máquina de estados, hoy orientada a TCP, con relay de datagramas o escucha de conexiones entrantes hacia el cliente. El estándar también define el método de autenticación **GSSAPI (0x01)**, la consigna del TPE lo menciona en el RFC 1928 pero aclara explícitamente que no es un requisito de entrega. En la implementación actual, el parser de greeting solo reconoce **0x02** (usuario/contraseña, RFC 1929) y **0x00** (sin autenticación): si el cliente ofrece ambos, se prioriza **0x02** y las credenciales se validan contra el **monitorstore**; si solo ofrece **0x00**, el servidor acepta y registra la sesión como usuario **Anon**

antes de pasar al request **CONNECT**. Cualquier otro método, incluido **0x01**, se ignora en la negociación y, si no queda ninguna opción compatible, el servidor responde **05FF** y cierra la conexión. Incorporar GSSAPI implicaría extender el greeting para seleccionarlo, añadir un nuevo estado en la máquina de estados con la sub negociación GSS-API (un intercambio de tokens que puede ser de varios mensajes) y, probablemente, delegar las llamadas bloqueantes a la librería GSS en un thread auxiliar con el mismo esquema usado hoy para **getaddrinfo**, sin reutilizar la tabla de usuarios de ChungusMonitor. Por esa complejidad y porque el objetivo del trabajo es demostrar autenticación usuario/contraseña, GSSAPI queda como extensión orientada a despliegues

A nivel operativo, convendría persistir en disco la tabla de usuarios y el **ACCESSLOG** (carga al arrancar y volcado atómico tras **ADD_USER**, **DEL_USER** y **SET_PASSWORD**), almacenar contraseñas como hashes en lugar de texto plano, y gobernar el acceso anónimo desde ChungusMonitor. En DNS, **getaddrinfo_a(3)** podría reemplazar el hilo auxiliar por consulta, complementado con un caché con TTL configurable. También se podría guardar en un archivo las configuraciones para que no sean datos volátiles, quizás aceptando una flag para elegir su path, permitiendo tener múltiples presets de configuración.

ChungusMonitor podría ampliarse con métricas adicionales (estadísticas DNS, desglose por usuario o destino, exportación del log) y el cliente con sub comandos más cómodos. El requisito de ≥ 500 clientes concurrentes se cumple con la implementación actual: el array `sessions[]` se redimensiona dinámicamente al subir `max_connections` mediante **CONFIG**, y el selector basado en `epoll` no impone restricciones de **FD_SETSIZE**. El límite superior es de 16384 sesiones (**STORE_SESSIONS_CAP_MAX**).

Por otro lado, las métricas que expone ChungusMonitor son instantáneas/snapshots del estado del servidor en el momento en que se ejecuta el comando. Una extensión sería implementar actualizaciones en tiempo real, por ejemplo que envíe periódicamente las estadísticas al cliente sin necesidad de consultas repetidas, o incorporando gráficos en tiempo real en la interfaz TUI.

En cuanto a la lista de denegación de hosts, la comparación exacta de cadenas (`strcmp`) obliga al administrador a agregar explícitamente cada subdominio, como ya se mencionó. Se podría implementar un mecanismo automático para bloquear todos los subdominios.

7. Conclusiones

Como conclusión del proyecto, se pudo desarrollar un servidor proxy SOCKSv5 completo junto con un protocolo de administración propio y su correspondiente cliente, partiendo de un servidor de eco básico. Este proceso permitió profundizar en la comprensión de conceptos fundamentales relacionados con protocolos de red, autenticación, resolución de nombres, manejo de errores y comunicación mediante entrada/salida no bloqueante.

Por otro lado, se destaca que la principal restricción de la consigna, consistente en implementar el servidor utilizando un único hilo con entrada/salida no bloqueante multiplexada, tuvo una influencia determinante en el diseño de la solución. Esta condición llevó a adoptar una arquitectura basada en eventos, con una clara separación entre la lógica de los protocolos, implementada mediante parsers y máquinas de estado, y el manejo de la entrada/salida, encargado de la transferencia de datos entre los distintos extremos de la comunicación.

Por último, la estrategia de desarrollo incremental resultó fundamental para la evolución del proyecto. Este enfoque permitió mantener una aplicación funcional durante todo el proceso de implementación, facilitar la incorporación progresiva de nuevas funcionalidades y desarrollar pruebas automatizadas para los componentes más críticos. Asimismo, la reutilización de la misma estructura de diseño en los servidores de eco, SOCKSv5 y monitoreo permitió obtener una solución más consistente, modular y fácil de mantener.

8. Guía de Instalación

8.1 Requisitos del Entorno

El proyecto está pensado para compilar y ejecutar en Linux . Las dependencias son:

```
sudo apt-get update  
  
sudo apt-get install -y build-essential gcc make check pkg-config
```

La biblioteca Check solo es necesaria para compilar y correr las pruebas automatizadas; en Ubuntu 24.04 el paquete se llama *check*.

8.2 Obtención del Código y Compilación

```
git clone https://github.com/SebaNLee/Protos-TPE-20261Q.git  
  
cd Protos-TPE-20261Q  
  
make clean all
```

Comandos disponibles del *makefile*:

Comando	Descripción
make all	Compila bin/server y bin/client
make server / make client	Compila un binario individual
make test	Compila las pruebas en build/
make check	Compila y ejecuta todas las pruebas
make clean	Elimina obj/, bin/ y build/

Dado que Make no rastrea dependencias de encabezados, tras modificar un archivo .h corresponde ejecutar:

```
make clean && make all
```

8.3 Entorno Docker

Para desarrollo en sistemas no Linux (por ejemplo macOS) el repositorio incluye un Dockerfile (Ubuntu 24.04, plataforma linux/amd64) y un docker-compose.yml que publica los puertos 1080 y 8080:

```
docker compose build

docker compose run --rm dev make clean all test check

docker compose run --rm --service-ports dev ./bin/server
```

8.4 Artefactos Generados

Ruta	Contenido
bin/server	Servidor proxy SOCKSv5 + monitoreo
bin/client	Cliente de monitoreo TUI
build/*_test	Binarios de pruebas
obj/	Objetos intermedios de compilación

9. Guía de Configuración

El servidor puede configurarse, mediante distintos flags al momento de su ejecución.

9.1 Argumentos del Servidor

```
./bin/server [-p <socks>] [-m <monitor>] -u <user>:<pass>... -a <admin>:<pass>...
```

Flag	Descripción	Valor por defecto
-p	Puerto de escucha del servicio SOCKSv5	1080
-m	Puerto de escucha del servicio de monitoreo	8080
-h	Imprime el mensaje de uso y termina	—
-u	Usuario SOCKS user:pass (repetible, al menos uno)	— (obligatorio)
-a	Usuario admin user:pass (repetible, al menos uno)	— (obligatorio)

Ambos sockets pasivos son IPv6 *dual-stack*, por lo que aceptan clientes IPv4 e IPv6 sin configuración adicional.

9.2 Argumentos del Cliente de Monitoreo

`./bin/client [-p <puerto>]`

Flag	Descripción	Valor por defecto
-p	Puerto del servicio de monitoreo	8080

Host fijo: 127.0.0.1. Login de administrador por prompts de la TUI.

Ambas aplicaciones procesan sus argumentos con `getopt(3)`, siguiendo las convenciones de utilidades de POSIX.

9.3 Parámetros en Tiempo de Ejecución

Vía menú Configuration del cliente, o comando CONFIG del protocolo (sección 2.2.6).

9.4 Usuario Inicial y Roles

Al arrancar, el servidor no crea usuarios por defecto. Los usuarios iniciales se definen exclusivamente por línea de comandos: **-u <usuario>:<contraseña>** agrega un usuario con rol **user** (puede autenticarse en el proxy SOCKS) y **-a <usuario>:<contraseña>** agrega un administrador (además puede autenticarse en ChungusMonitor). Ambos flags son obligatorios al menos una vez y pueden repetirse para cargar varios usuarios.

Existen dos roles:

- **user**: puede autenticarse en el proxy SOCKS.
- **admin**: además puede autenticarse en el puerto de monitoreo y administrar el servidor.

En tiempo de ejecución, los usuarios se gestionan desde el cliente TUI (opciones Add user, Delete user y Change password) o, de forma equivalente, con los comandos del protocolo **ADD_USER**, **DEL_USER** y **SET_PASSWORD**. El sistema impide eliminar al último administrador, de modo que el servidor nunca quede sin acceso de administración.

10. Ejemplos de Prueba

10.1 Pruebas Automatizadas

El proyecto cuenta con binarios de prueba basados en la biblioteca Check, que se compilan y ejecutan con ***make check***:

Prueba	Alcance
buffer_test, selector_test, parser_test, parser_utils_test, netutils_test, stm_test	Módulos compartidos de la cátedra
echo_test	Servidor de echo no bloqueante
greeting_test, auth_test, request_test	Parsers SOCKS (negociación, RFC 1929, CONNECT con IPv4/FQDN y comandos inválidos)
store_test	Estado compartido entre usuarios, configuración, sesiones y registro de accesos
monitor_commands_test	Intérprete de comandos ChungusMonitor (syntaxis, autenticación y errores)
monitor_test	Integración TCP del monitoreo: saludo, autenticación, pipelining, half-close y alta de usuarios
socks_partial_read_test	Handshake SOCKS con lecturas parciales (greeting y auth en un solo segmento, y byte a byte con 200 ms de demora)

10.2 Pruebas Funcionales del Proxy SOCKS

Con el servidor en ejecución, el funcionamiento se verifica con curl usando el proxy:

Destino por FQDN (resolución DNS en el servidor)

`curl --socks5-hostname admin:admin@localhost:1080 http://example.com`

```
MacBook-Air-8:~ geronimo$ curl --socks5-hostname admin:admin@localhost:1080 http://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" c
ontent="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-
ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domai
n</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a
href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
MacBook-Air-8:~ geronimo$
```

Destino por FQDN resuelto por curl (el proxy recibe la IP)

`curl --socks5 admin:admin@localhost:1080 http://example.com`

```
MacBook-Air-8:~ geronimo$ curl --socks5 admin:admin@localhost:1080 http://example.com
curl: (97) Could not resolve host: example.com
MacBook-Air-8:~ geronimo$
```

Credenciales inválidas: curl reporta el rechazo de autenticación

`curl --socks5 admin:incorrecta@localhost:1080 http://example.com`

```
MacBook-Air-8:~ geronimo$ curl --socks5 admin:incorrecta@localhost:1080 http://example.com
curl: (97) User was rejected by the SOCKS5 server (1 1).
MacBook-Air-8:~ geronimo$
```

Destino inexistente: el proxy responde host inalcanzable (REP 0x04)

`curl --socks5-hostname admin:admin@localhost:1080 http://dominio-inexistente.invalid`

```
MacBook-Air-8:~ geronimo$ curl --socks5-hostname admin:admin@localhost:1080 http://dominio-inexistente.invalid
curl: (97) Can't complete SOCKS5 connection to dominio-inexistente.invalid. (4)
MacBook-Air-8:~ geronimo$
```

Puerto cerrado: el proxy responde conexión rechazada (REP 0x05)

`curl --socks5 admin:admin@localhost:1080 http://localhost:1`

```
MacBook-Air-8:~ geronimo$ curl --socks5 admin:admin@localhost:1080 http://localhost:1
curl: (97) Can't complete SOCKS5 connection to localhost. (5)
MacBook-Air-8:~ geronimo$
```

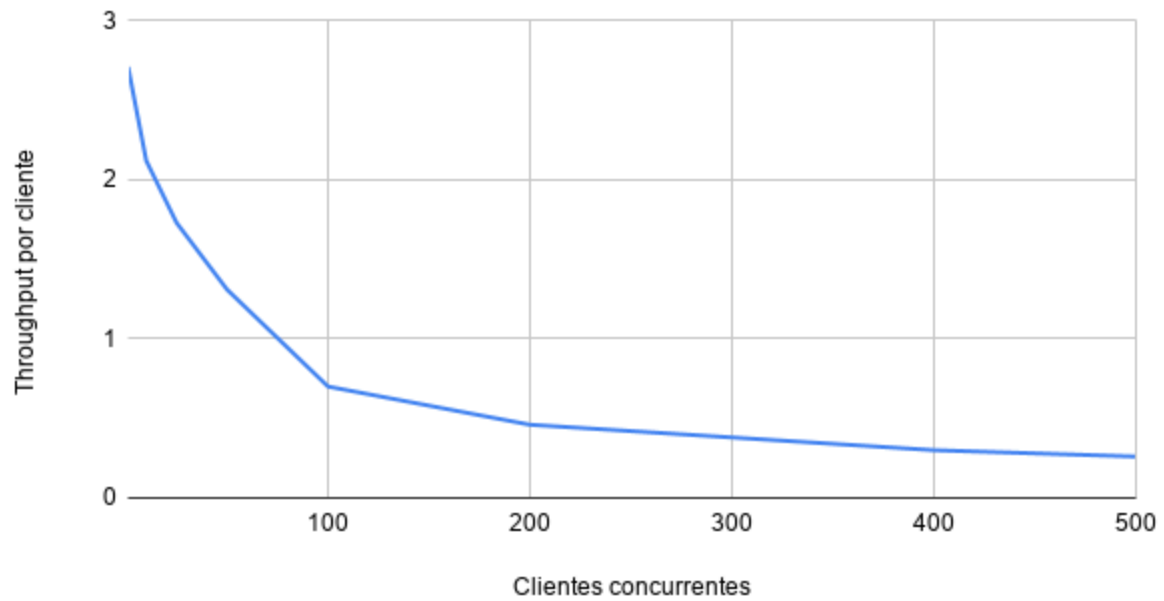

10.3 Pruebas de Estrés y Throughput

Para evaluar el rendimiento se desarrolló un generador de carga en C (*stress_client*) que implementa el handshake SOCKS5 completo. El modo *connections* abre N túneles hacia un *echo_backend* local y los mantiene idle 1 s antes de cerrarlos. El modo *throughput* transfiere 64 KiB por cliente en ping-pong de 4 KiB (write/read por chunk) para evitar deadlock en el túnel. El modo *live* mantiene túneles abiertos mientras el operador consulta el monitor (*bin/client*: STATS, CONNECTIONS) en otra terminal.

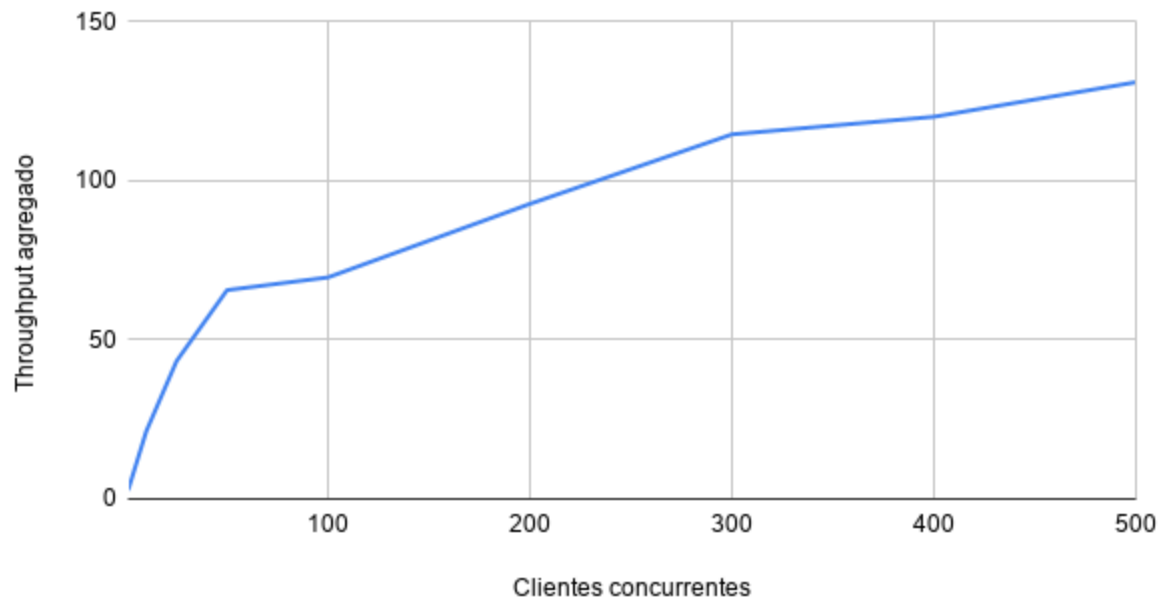
En escalabilidad (default *max_connections=1024*), se probó de 10 a 600 túneles sin fallos; el tiempo creció ~linealmente (~1 s para 10, ~10,4 s para 600). Con *CONFIG max_connections 16384*, una prueba puntual alcanzó 16.384/16.384 conexiones simultáneas en ~267 s (tope arquitectónico 2^{14}).

En throughput, la tasa de transferencia agregada sube hasta ~100–118 MiB/s (estabilizado ~100+ MiB/s con $N \geq 100$); la tasa por cliente por cliente baja (~3,0 MiB/s con $N=1$, ~0,24 MiB/s con $N=500$), coherente con un servidor single-thread multiplexado con *epoll*. No hubo errores ni pérdida de datos en las corridas documentadas.

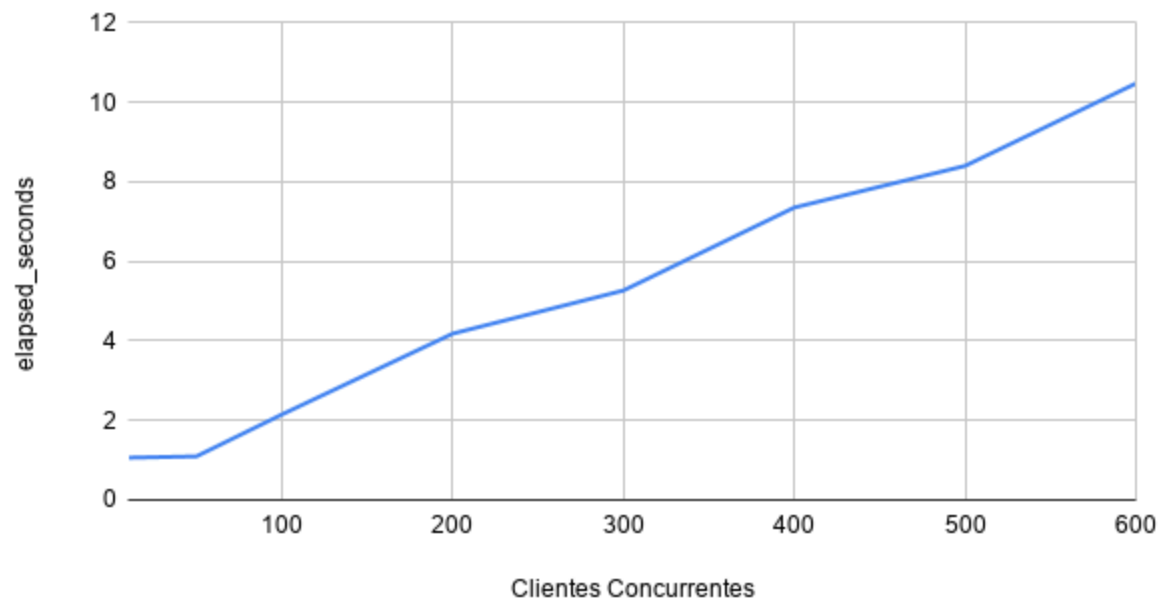
Clientes concurrentes vs Throughput por cliente



Clientes concurrentes vs Throughput agregado



Cientes concurrentes vs Segundos Transcurridos



11. Ejemplos de Configuración y Monitoreo

Los siguientes ejemplos muestran sesiones reales de administración utilizando la aplicación cliente. En cada caso se indica el comando ejecutado y la salida que produce (las cifras dependen del estado del servidor).

Menú Principal

```
ChungusMonitor v1.0 – 127.0.0.1:8080
User: admin

Stats
Active connections
Registered users
Configuration
> Add user
Delete user
Change password
Access log
Deny host
Deny IP
Undeny
Deny list
Quit

[Enter] Select  [↑↓] Navigate  [q] Quit
```

Figura 1: Menú Principal

Gestion de usuarios

```
ChungusMonitor v1.0 - 127.0.0.1:8080  
  
Username: admin  
Password:
```

Figura 2: Saludo del servidor y autenticación del usuario

```
ChungusMonitor v1.0 - 127.0.0.1:8080  
User: admin  
  
Add user  
  Username: chungus  
  Password:  
Admin? (y/n): y  
  
  User created o7  
  
Press Enter to continue...
```

Figura 3: Agregar un usuario nuevo

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Change password
  Username: chungus
  New password:

  Password updated o7

Press Enter to continue...
```

Figura 4: Cambiar la contraseña de un usuario existente

```
root@ca970ac78525:/work# ./bin/client
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Delete user
  Username: user
  Delete 'user'? (y/N): y

  User deleted o7

Press Enter to continue...
```

Figura 5: Eliminar un usuario

Registro de Accesos y Sesiones Activas

Los siguientes resultados se realizaron con dos usuarios activos en simultáneo.

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Registered users
  admin (admin)
  chungus (admin)

Press Enter to continue...
```

Figura 6: Usuarios activos conectados al servidor.

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Stats
  Total connections:      48
  Active connections:    29
  Bytes uploaded:        1600404
  Bytes downloaded:      13058338

Press Enter to continue...
```

Figura 7: Estadísticas del servidor

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin
```

Active connections

```
Anon play.google.com:443 RELAY 21127 7692
Anon beacons.gcp.gvt2.com:443 RELAY 2583 5753
Anon google.com:443 RELAY 3087 6766
Anon lh3.google.com:443 RELAY 3679 8681
Anon ogads-pa.clients6.google.com:443 RELAY 4073 11486
Anon lh3.googleusercontent.com:443 RELAY 3532 11785
Anon play.google.com:443 RELAY 4232 6454
Anon www.youtube.com:443 RELAY 491841 678842
Anon i.ytimg.com:443 RELAY 5466 353455
Anon googleads.g.doubleclick.net:443 RELAY 4309 7373
Anon yt3.ggpht.com:443 RELAY 4335 90814
Anon www.google.com.ar:443 RELAY 4783 5655
Anon fonts.gstatic.com:443 RELAY 2991 30500
Anon www.gstatic.com:443 RELAY 3105 6348
Anon tpc.googlesyndication.com:443 RELAY 2995 99302
Anon rr3---sn-uxaxjvh5gbxoupo5-x1xe7.googlevideo.com:443 RELAY 7757 3609832
Anon rr3---sn-uxaxjvh5gbxoupo5-x1xe7.googlevideo.com:443 RELAY 3402 885
Anon accounts.youtube.com:443 RELAY 11321 10224
Anon static.doubleclick.net:443 RELAY 3395 5473
Anon rr3---sn-uxaxjvh5gbxoupo5-x1xe7.googlevideo.com:443 RELAY 16733 3835629
Anon googleads.g.doubleclick.net:443 RELAY 2608 5906
Anon tpc.googlesyndication.com:443 RELAY 3007 5227
Anon pagead2.googlesyndication.com:443 RELAY 3596 5799
```

```
Press Enter to continue...
```

Figura 8: Lista de conexiones activas

Cambios de Configuración en Tiempo de Ejecución

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

> Max idle time (seconds)          [0]
   Max concurrent conns           [1024]
   I/O buffer size (bytes)        [4096]
   Back

[Enter] Select  [↑↓] Navigate  [q] Back
```

Figura 9: Sub-menú de configuración

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Max idle time (seconds)
  New value: 1000

Configuration updated o7

Press Enter to continue...
```

Figura 10: Configurar TTL (por default está desactivado)

Deny List

```
root@ca970ac78525:/work# ./bin/client
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Deny IP
  IP address: 10.11.25.1

  IP denied o7

Press Enter to continue...
```

Figura 11: Agregar una IP a la Deny List

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Deny host
  Hostname: www.google.com

  Host denied o7

Press Enter to continue...
```

Figura 12: Agregar un host a la Deny List

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Denied hosts and IPs
  Denied hosts:
  www.google.com
  Denied IPs:
  110.11.25.1

Press Enter to continue...
```

Figura 13: Listado de hosts e IPs en la Deny List

```
ChungusMonitor v1.0 - 127.0.0.1:8080
User: admin

Undeny
  Hostname or IP: 110.11.25.1

  Rule removed o7

Press Enter to continue...
```

Figura 14: Quitar una IP o hostname de la Deny List

12. Documento de Diseño del Proyecto

El sistema es un único proceso con un único hilo principal. Dos sockets pasivos, SOCKS y monitoreo, conviven en el mismo selector; cada conexión aceptada registra su propio descriptor con manejadores específicos del servicio. El único punto donde aparecen hilos adicionales es la resolución DNS, y su única interacción con el resto del sistema es depositar el resultado y despertar al selector.

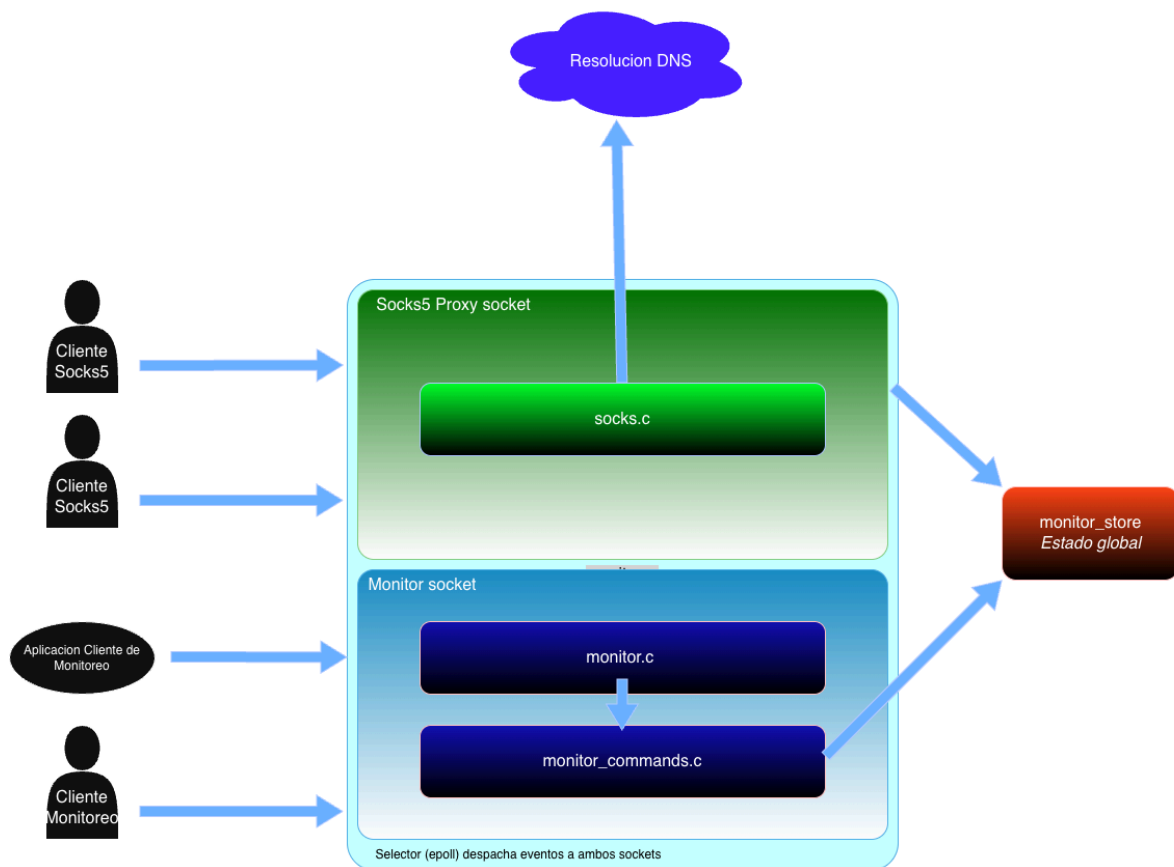


Figura 10: Diagrama de diseño del proyecto

13. Respuestas al feedback recibido de la pre-entrega

Se redacta esta sección adicional para expandir sobre algunas de las decisiones tomadas a partir del feedback recibido de la pre-entrega, las no mencionadas acá asumir que las tomamos. Vamos citando varios de los puntos mencionados en la devolución para ir explicándolas individualmente:

“¿Por qué usaron buffers de 4096 bytes? (no está mal; si justifican sus elecciones buscar defaults de otras herramientas o realizando pruebas de performance con distintos tamaños para encontrar el óptimo les sumaría puntos)”

Aclarado en la sección 2.1.

“Los registros de acceso los pueden imprimir hacia stdout y listo, no precisan guardarlos en un buffer circular (es un patrón común imprimir logs a stdout y que la aplicación se abstraiga de a dónde van esos logs) a menos que los estén usando para otra cosa”

Acá decidimos hacer ambas cosas. Los buffers circulares permiten a los admins ver el historial de conexiones desde el cliente TUI directamente, sin necesidad de tener acceso a la terminal que se ejecutó el servidor.

14. Repositorio

<https://github.com/SebaNLee/Protos-TPE-20261Q>

15. Bibliografía

1. RFC 1928 — *SOCKS Protocol Version 5*. <https://datatracker.ietf.org/doc/html/rfc1928>
2. RFC 1929 — *Username/Password Authentication for SOCKS V5*.
<https://datatracker.ietf.org/doc/html/rfc1929>
3. Canal de YouTube Jacob Sober:
https://www.youtube.com/playlist?list=PL9IEJIKnBJjH_zM5LnovnoaKIXML5qh17
4. <https://man7.org/linux/man-pages/man7/epoll.7.html>
5. <https://man7.org/linux/man-pages/man3/termios.3.html>
6. https://nginx.org/en/docs/http/nginx_http_proxy_module.html